

ShopSense

E-Commerce Search & Recommendation System

Project Abstract — Architecture, Flow, Data Structures, Database, APIs, Security

Built with Node.js + SQLite · zero runtime dependencies · hybrid search + hybrid recommender

This document explains the ShopSense project end to end in simple language: what problem it solves, who it is for, how it is built, how data flows from the browser to the database and back, which algorithms and data structures power it, what the database looks like, the full API surface, and the security measures in place.

Version: 1.0 (single source of truth: the running repository)

Runtime: Node.js 22.5+ (uses the built-in node:sqlite module)

Dependencies: None at runtime and none at build time

Data stores: SQLite database + in-memory search/recommendation indexes

1. Executive Summary

ShopSense is a complete, working e-commerce discovery platform. Instead of showing a shopper 10,000 raw options, it surfaces the best few and explains why. It combines two engines:

- A hybrid Search engine: keyword ranking (BM25), semantic-ish vector retrieval (TF-IDF + cosine), typo correction, autocomplete, faceted filters and Google-style recall with 100+ ranked results per query.
- A hybrid Recommender: trending (time-decayed popularity), content-based similarity, item-item collaborative filtering, matrix factorization, plus a signal learned from the user's own search history.

Around these engines it provides a real backend: user accounts (register/login with JWT), a persistent wishlist, a shopping cart, orders/checkout, product specifications and aggregated reviews, curated ranked lists, an admin API, and an affiliate redirect that sends Buy clicks to the correct merchant. Everything is persisted in SQLite so data survives restarts.

The entire system is implemented with Node.js built-ins only (node:http, node:sqlite, node:crypto). There is no framework and no third-party package to install, which makes it portable, fast to start, and easy to audit.

2. Problem Statement

Modern shoppers are overwhelmed. A single search returns thousands of near-identical items, most irrelevant; ratings are scattered across many shopping sites; and new visitors get no personalisation because the system knows nothing about them. Small demos usually stop at “a list of products”, with no persistence, no accounts, no explainability and no way to scale the matching logic.

The problem ShopSense addresses:

- Relevance: find the right product quickly, tolerate typos, and understand why a result appeared.
- Discovery: recommend products using behaviour (trending, similar, also-viewed, personalised) and the user's own searches.
- Trust and utility: show real specifications and reviews aggregated across stores, and compare prices.

- Identity: let a visitor browse freely, but require a profile for meaningful actions (wishlist, cart, checkout), then remember their data.
- Engineering: demonstrate a clean, layered, dependency-free architecture that maps to production designs (search cluster, vector store, relational DB) while still running locally with one command.

3. Target Users

User	Needs	What ShopSense gives them
Shopper (guest)	Browse and search without friction	Full catalogue browsing, search, product pages, reviews, price comparison
Registered shopper	Save items and get personalisation	Profile, wishlist, cart, orders, recommendations from activity + search history
Admin / merchandiser	Manage catalogue and see performance	Product CRUD, catalogue import, user/order listing, analytics dashboards
Developer / evaluator	Understand and extend the system	Layered code, documented algorithms, tests, zero-setup run, this abstract

4. Goals — Current and Future

4.1 Current goals (already implemented)

- Hybrid search: BM25 + TF-IDF vector retrieval fused with RRF, typo tolerance, autocomplete, facets.
- Google-style recall: every real query returns 100+ results — precise matches ranked first, then relevant catalogue backfill, paginated.
- Hybrid recommendations with human-readable reasons and cold-start handling.
- Persistent accounts, wishlist, cart, orders and interactions in SQLite; seeds 579 deduplicated products.
- Product specifications and reviews aggregated across stores; multi-store price comparison.
- Affiliate-aware Buy links (exact brand + title deep links, tracking parameters, optional product-link resolver).
- Clean categorisation into storefront verticals and a home page with top-brand and per-category shelves.
- Full REST API, a test suite (65 tests) and a documented, dependency-free architecture.

4.2 Future goals

- Swap SQLite for PostgreSQL + pgvector and the hand-rolled index for OpenSearch/Elasticsearch.
- Replace TF-IDF with neural embeddings and an approximate-nearest-neighbour index (HNSW).
- Move batch generation/training to a scheduler or queue and cache results in Redis.
- Add OAuth/refresh tokens, real ingestion connectors, and real affiliate APIs (Amazon PA-API, etc.).
- Learning-to-rank from clicks, A/B testing, and session/sequence recommenders.

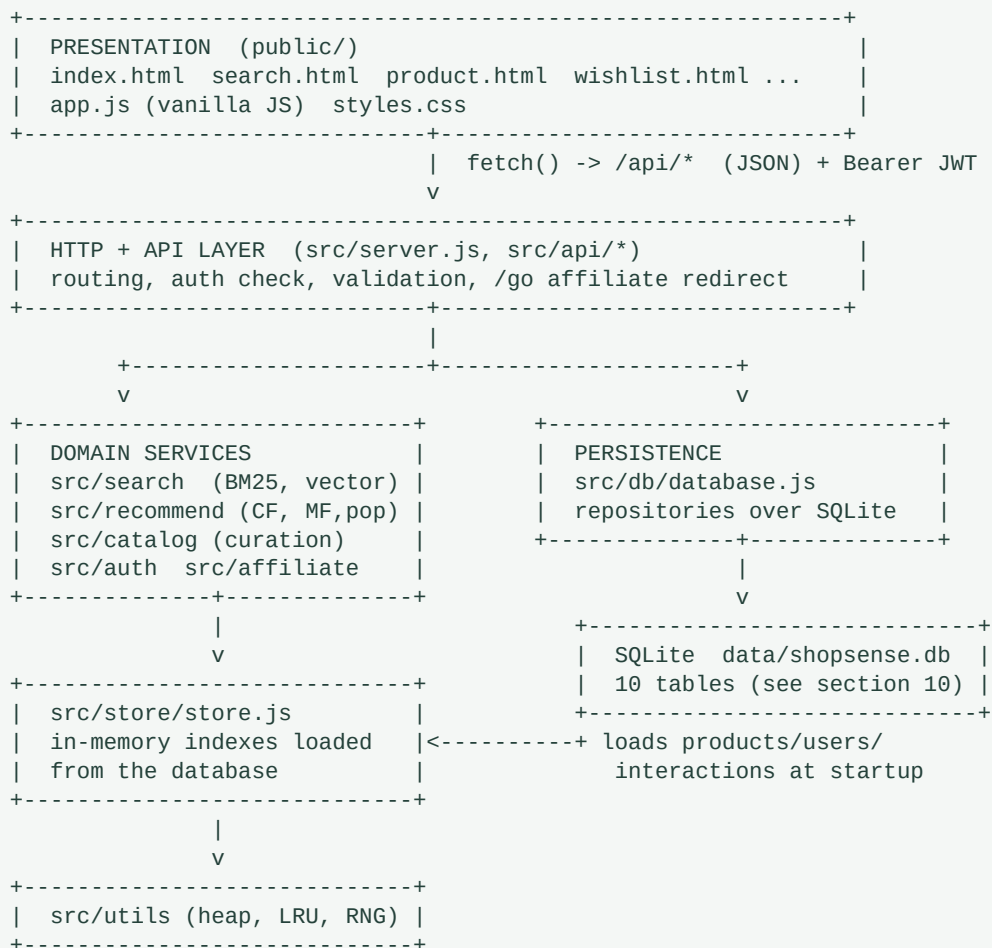
5. Tech Stack (in detail)

Layer	Technology	Why it is used
Runtime	Node.js 22.5+	Single language for the whole backend; built-in sqlite/crypto/http
HTTP server	node:http (no framework)	Hand-rolled router over the built-in server keeps it dependency-free

Database	SQLite via node:sqlite (DatabaseSync)	Embedded, file-based, transactional; no server to install; WAL mode
Auth/crypto	node:crypto	scrypt password hashing, HMAC-SHA256 JWTs, timing-safe compare, random ids
Frontend	Vanilla JavaScript + HTML5 + CSS3	No build step; talks to the API with fetch(); renders pages client-side
Client state	localStorage	Stores the JWT and the current user profile
Product imagery	Real product photos + generated SVG fallback	Real images from data sources; local SVG illustrations offline
Data sources	DummyJSON, Open Beauty Facts, synthetic generators	Real products/images plus deterministic branded supplements
Testing	node:test (built-in)	Zero-dependency unit and API tests

6. System Architecture

ShopSense is a classic layered (n-tier) architecture. Each layer only talks to the layer below it, which makes the system easy to reason about and to replace piece by piece.



Key idea: the search and recommendation engines keep their indexes in memory for speed (an inverted index, TF-IDF vectors, a trie, item-item neighbours, latent factors). SQLite is the source of truth for everything that changes (accounts, cart, orders, wishlist, events). At startup the store reads the catalogue

from SQLite and rebuilds the in-memory indexes.

7. Project Flow

7.1 Offline / build-time pipeline (runs at startup)

```
scripts/generate-catalog.js / fetch-catalog.js -> data/products.json
scripts/generate-extra.js                      -> data/extra-products.json
scripts/generate-interactions.js                -> data/users.json + interactions.json
    (dedupe + categorise) -> SQLite tables (products, users, interactions, ...)
                           -> build in-memory search index (inverted index,
                           TF-IDF vectors, trie)
                           -> train recommender (popularity, item-item CF,
                           matrix factorization)
```

7.2 Online request lifecycle

- The browser calls a REST endpoint, e.g. GET /api/search?q=running+shoes, using fetch().
- src/server.js receives the request and, for /api/*, forwards it to src/api/routes.js.
- routes.js builds a context (the store, the database repo, and the authenticated user from the JWT) and dispatches to a sub-router (auth, cart, orders, admin, wishlist) or to the built-in search/recommend handlers.
- Search handlers call the in-memory SearchService (BM25 + vectors + RRF + backfill) and read facets from the index.
- Write endpoints (cart, wishlist, orders, events, admin product changes) call repositories that run parameterised SQL against SQLite, then refresh the relevant in-memory state.
- The response is serialised to JSON and returned; the frontend renders cards, grids, tables and charts.

7.3 Gated-interaction flow (guest to profile)

- A guest clicks the wishlist heart. The frontend sees there is no token and opens the sign-up/login modal instead of calling the API.
- On successful register/login the API returns a JWT and the user object. The frontend stores them, refreshes the wishlist state, and automatically resumes the pending action (the item is added).
- From then on, fetch() attaches Authorization: Bearer <token> automatically, and cart/wishlist/order endpoints accept the request.

8. Frontend, Backend and Database — How They Connect

Concern	Who does it	How
Rendering	Frontend	app.js builds DOM with template strings; no framework
Data access	Frontend	fetch('/api/...') with JSON bodies; Bearer token in the header
Session	Frontend	JWT + user stored in localStorage; GET /api/auth/me restores the session on load
Routing	Backend	node:http server; a small table maps URL prefixes to route modules
Business logic	Backend	SearchService, RecommendationService, catalog helpers
Persistence	Backend	Repositories in src/db/database.js using prepared statements
Storage	Database	SQLite file data/shopsense.db (WAL mode, foreign keys on)

Round trip example — adding to the cart:

```

Browser:  fetch('/api/cart', {method:'POST',
          headers:{Authorization:'Bearer <jwt>', 'Content-Type':'application/json'},
          body: JSON.stringify({productId:'x_12', quantity:2}) })
|
Server:   handleApi -> cartRoutes.handle -> repo.cart.add(userId, productId, 2)
|
SQLite:   INSERT INTO cart_items(user_id, product_id, quantity, added_at)
          VALUES(?,?,?,?) ON CONFLICT(user_id,product_id)
          DO UPDATE SET quantity = quantity + excluded.quantity;
|
Server:   reads the cart back, joins each row with the in-memory product
          document, computes line totals and subtotal -> JSON
|
Browser:  re-renders the cart / wishlist and repaints the heart state

```

9. Data Structures & Algorithms (DSA) Architecture

This section is the heart of the project. For each data structure or algorithm it explains where it lives, what it does, how it works, a concrete example, and its cost.

9.1 Inverted Index — term to documents

Where: src/search/invertedIndex.js, built by src/search/searchService.js

What: A hash map from each indexed term to a posting list (documents containing it).

How: Every product's text (title, brand, category, tags, description) is tokenised; each token is inserted into the index. A query looks up only the terms it contains instead of scanning the whole catalogue.

Example: searching hair oil looks up postings('hair') = {p12, p40, ...} and postings('oil') = {p12, p77, ...}; only the union is scored. Cost: $O(1)$ term lookup, $O(\text{sum of postings})$ to score.

9.2 BM25 — relevance ranking

Where: src/search/bm25.js

What: The industry-standard keyword ranking function that weights rare terms more and normalises for document length.

How: $\text{score}(D, Q) = \text{sum over query terms } t \text{ of } \text{IDF}(t) * (\text{tf} * (k_1 + 1)) / (\text{tf} + k_1 * (1 - b + b * |D| / \text{avgdl}))$.

Example: for the query running shoes, “running” is rarer than “shoes”, so it gets a higher IDF and products that contain it rank higher; a very long description is not automatically better because of the length normalisation. Cost: $O(\text{terms} \times \text{postings})$.

9.3 TF-IDF Vectors + Cosine Similarity — semantic retrieval

Where: src/search/vectorModel.js

What: Represents each product and the query as a sparse vector of weighted terms, then measures the angle between them.

How: Vectorise into term -> weight, L2-normalise, and take the dot product (equivalent to cosine similarity for unit vectors). This catches related wording that exact keywords miss.

Example: a query smartwatch and a product described as a wearable fitness watch share vocabulary and score highly even if the literal word “smartwatch” is absent. Cost: $O(\text{non-zero terms})$.

9.4 Reciprocal Rank Fusion (RRF) — merging result lists

Where: src/search/searchService.js (rrfFuse)

What: Combines the BM25 list and the vector list into one ranking without needing their scores to be comparable.

How: For each document, add $1/(k + \text{rank})$ across the lists it appears in, weighted per list; higher fused score wins.

Example: a product that is rank 1 in BM25 and rank 3 in vectors gets $1/(60+1) + 0.9 * 1/(60+3)$, beating one that is rank 1 in only one list. This is why the search strategy is labelled “hybrid”.

9.5 Trie (Prefix Tree) — autocomplete

Where: `src/search/trie.js`, used by `GET /api/search/suggest`

What: A tree keyed by characters; each node stores child letters and a weight.

How: Walk down the characters of the typed prefix, then collect the sub-tree's words ordered by weight (brand/category words weighted higher, titles lower).

Example: typing "sho" returns shoes, shirt, shorts. Cost: $O(m)$ to reach the prefix node plus the size of the matching sub-tree.

9.6 Levenshtein Distance (DP) — typo correction

Where: `src/search/fuzzy.js` (`levenshtein`, `nearestTerms`, `correctQuery`)

What: The minimum number of single-character edits (insert/delete/substitute) to turn one word into another, computed with dynamic programming over an $(n \times m)$ table.

How: For an unknown query word, the engine scans the vocabulary and picks the nearest term within a small edit distance, then rewrites the query.

Example: `sunscren` is distance 1 from `sunscreen`, so the search corrects it and still finds the right products. Cost: $O(n \times m)$ per candidate.

9.7 Binary Min-Heap + Top-K — efficient "best N"

Where: `src/utls/minheap.js` (class `MinHeap`, function `topK`), used by the recommender and collaborative filtering

What: A binary heap that keeps only the K best items while streaming through many candidates.

How: Push each item; when the heap exceeds size K, pop the smallest. The result is the K largest.

Example: personalising a feed scores thousands of products but only the top 10 are needed — `topK` is $O(n \log k)$ instead of $O(n \log n)$ for a full sort.

9.8 LRU Cache — bounded hot-data cache

Where: `src/utls/lru.js` (class `LRUCache`)

What: A map that evicts the least-recently-used entry when it grows past a limit, giving $O(1)$ get and put.

How: JavaScript Map preserves insertion order, so a get re-inserts the key at the end (most recent) and overflow deletes the first key (oldest).

The utility is included in the codebase and is the natural place to cache hot query results; the current search path recomputes results, keeping the memory footprint tiny and results always fresh.

9.9 Time-Decayed Popularity — trending

Where: `src/recommend/popularity.js`

What: A popularity score where recent interactions count more than old ones.

How: Each interaction contributes a weight (view 1, click 2, wishlist 3, cart 4, purchase 5) multiplied by $\exp(-\lambda \times \text{ageDays})$, with a 21-day half-life.

Example: a product bought by five people yesterday outranks one viewed by fifty people last year. Cost: $O(\text{interactions})$.

9.10 Item-Item Collaborative Filtering

Where: `src/recommend/itemItemCF.js`

What: "Users who viewed this also viewed" — similarity between products based on shared user behaviour.

How: Build a matrix item $\times$ user of interaction weights, compute cosine similarity between item vectors, and keep the top-K neighbours per item.

Example: because many users who view running shoes also view cushioned insoles, the two items become neighbours and are recommended together. Neighbour computation is done once at startup.

9.11 Matrix Factorization (SGD) — personalisation

Where: `src/recommend/matrixFactor.js`

What: Learns small latent vectors for each user and item so their dot product predicts preference.

How: Train with stochastic gradient descent on observed interactions plus sampled negatives (factors=16, epochs=30, learning rate/regularisation), minimising prediction error.
Example: the model may discover a latent “budget skincare” dimension shared by a user and certain products, and recommend them even with no exact tag overlap. Cost: O(epochs x interactions x factors).

9.12 Hybrid Fusion in the Recommender

Where: src/recommend/recService.js

What: Blends several signals into one score and explains the winner.

How: Weighted sum: collaborative filtering 0.4, matrix factorization 0.3, content 0.2, popularity 0.1, plus a search-history signal 0.35. Each list is min-max normalised first. New users fall back to a cold-start path built from popularity and their stated preferences.

Example: the returned reason “Based on your recent searches” appears when the user's own recent queries (search history) contributed most to that item.

9.13 Hashing, Deterministic RNG and Search History

- Stable keys: FNV-1a hashing (and a small hash in the data generators) turns a product id into a repeatable number, so generated images, reviews and specs are the same on every request.
- Deterministic RNG: mulberry32 (src/utls/rng.js) produces a reproducible pseudo-random stream for seeding the catalogue and synthetic data.
- Search history: each authenticated search is stored; the recommender turns recent queries into a weighted, content-expanded score map (searchProfile) blended into the personalised feed.

10. Database Design and ER Diagram

ShopSense uses SQLite with 10 tables. The catalogue is stored both as queryable columns (id, title, brand, category, price, rating...) and as a full JSON document in the data column, which lets the schema stay stable while product attributes evolve. Prepared statements (parameterised queries) are used everywhere.

10.1 Entities

#	Table	Primary key	Purpose
1	products	id	Catalogue: indexed columns + full product document (JSON)
2	users	id	Accounts: credentials, role, optional phone/profession, is_seed flag
3	interactions	id	Implicit feedback (view/click/wishlist/cart/purchase) for the recommender
4	wishlist_items	user_id, product_id	Saved products per user
5	cart_items	user_id, product_id	Active cart with quantity
6	orders	id	Placed orders (status, total, item count)
7	order_items	order_id, product_id	Line items of each order
8	events	id (autoincrement)	Runtime event log (views, clicks, wishlist/cart/purchase)
9	search_logs	id (autoincrement)	Global search analytics (query, result count, clicked id)
10	search_history	id (autoincrement)	Per-user recent queries that drive recommendations

10.2 Relationships and cardinality

- users 1 : N orders, cart_items, wishlist_items, interactions, events, search_history
- products 1 : N order_items, cart_items, wishlist_items, interactions, events

- orders 1 : N order_items (ON DELETE CASCADE)
- search_logs is a standalone analytics log (no foreign keys)

Most transactional tables also carry a logical reference to products by id even where no SQL foreign key is declared, because the catalogue is regenerated and the engine joins rows to the in-memory product map.

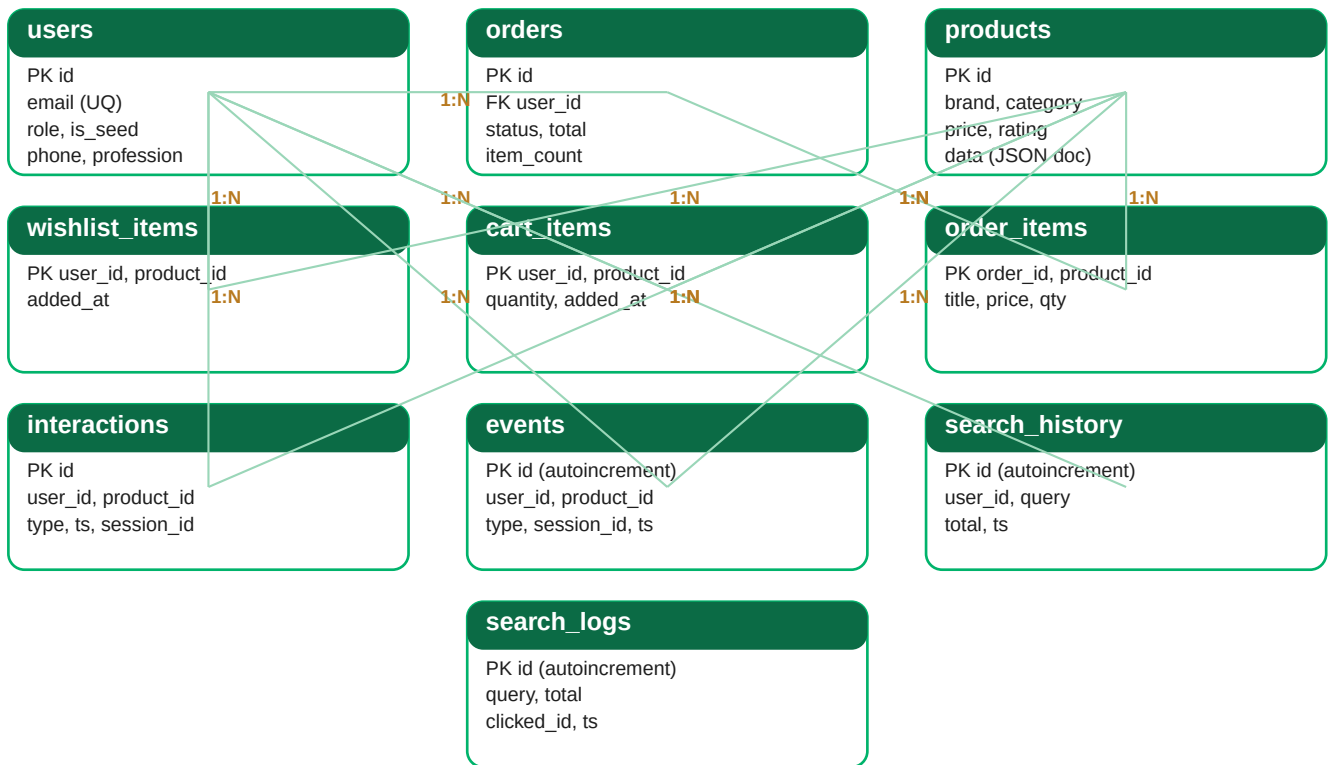
10.3 Indexes

- products: category, brand, price
- users: unique email
- interactions: user_id, product_id
- orders: user_id; events: product_id, user_id; search_history: user_id

10.4 Real ER diagram

ShopSense — Entity-Relationship Diagram

10 tables. 1:N relationships shown with crow's-foot style lines and cardinality labels (1 / N).



Legend: PK = primary key, FK = foreign key, UQ = unique. products.id is referenced logically by the transactional tables; orders -> order_items uses ON DELETE CASCADE.

11. API Reference (all endpoints)

Base URL: <http://localhost:3000/api>. “Auth” means the request must carry Authorization: Bearer <JWT>. “Admin” means the token must belong to an admin. “Guest” means no token is required.

Catalogue, search and discovery (guest)

Method	Path	What it does
GET	/health	Liveness + product count
GET	/home	Top-brand shelf and per-category shelves for the home page
GET	/categories	Leaf categories with counts and brands
GET	/users	Registered customers (generated demo accounts hidden)
GET	/search?q=&vertical=&brand=&priceMin=&priceMax=&minRating=&sort=&page=&pageSize=	Hybrid search with Google-style recall (100+ results), facets and pagination
GET	/search/suggest?q=	Autocomplete suggestions from the trie
GET	/products/:id	Full detail: specs, reviews, offers, similar, also-viewed, ranked-in
GET	/products/:id/specs	Grouped product specifications
GET	/products/:id/reviews?source=&minRating=&sort=	Aggregated reviews + rating summary
GET	/products/:id/similar	Content-based similar products

GET	/products/:id/recommendations	Item-item collaborative filtering
GET	/products/:id/prices	Multi-store offers, lowest first
GET	/recommendations/trending?category=&limit=	Time-decayed trending
GET	/recommendations?userId=&limit=	Personalised feed (search-history + CF/MF); trending if no user
GET	/curated-lists	Curated ranked-list index
GET	/curated-lists/:id	A ranked list with products and scores
GET	/images/:id.svg	Deterministic SVG product illustration (offline fallback)
GET	/stats	Search and recommender analytics for the dashboard
POST	/events	Log an interaction {productId, type, userId}
POST	/ratings	Submit a rating {productId, value}

Authentication

Method	Path	What it does
POST	/auth/register	Create an account {name, email, password, phone?, profession?} -> {token, user}
POST	/auth/login	Log in {email, password} -> {token, user}
GET	/auth/me	Auth: return the current user

Wishlist (Auth) and Cart (Auth)

Method	Path	What it does
GET	/wishlist	Saved products for the current user
POST	/wishlist	Add {productId}
DELETE	/wishlist/:productId	Remove an item
GET	/me/wishlist	Alias of GET /wishlist
GET	/cart	Cart with product documents and totals
POST	/cart	Add {productId, quantity}
PATCH	/cart/:productId	Set quantity (0 removes)
DELETE	/cart/:productId	Remove one item
DELETE	/cart	Clear the cart

Orders (Auth)

Method	Path	What it does
POST	/orders	Checkout the cart -> creates an order and clears the cart
GET	/orders	Order history for the current user
GET	/orders/:id	Order detail (owner or admin)

Admin (Admin)

Method	Path	What it does
--------	------	--------------

GET	/admin/products?q=&limit=&offset=	List / search products
POST	/admin/products	Create a product
PUT	/admin/products/:id	Update a product
DELETE	/admin/products/:id	Delete a product
POST	/admin/catalog/import	Bulk upsert {products:[...]} or regenerate
GET	/admin/users	List registered customers
GET	/admin/orders	List all orders
PATCH	/admin/orders/:id	Update order {status}
GET	/admin/analytics	Catalogue, orders, events and search analytics

Non-API route

Method	Path	What it does
GET	/go?product=&merchant=&userId=	Affiliate redirect to the correct merchant (host-validated) and logs the click

The API is also connected to external data sources during seeding: DummyJSON and Open Beauty Facts provide real products and images, and the optional affiliate product-link resolver maps a product to a real merchant URL. Amazon PA-API / Rainforest / a custom gateway can be plugged in through the resolver hook.

12. Security

Area	Measure	Where
Passwords	bcrypt salted hashing (never stored in plain text); timing-safe verification	src/auth/auth.js
Sessions	Stateless JWT signed with HMAC-SHA256; expiry claim; secret from env or a .env file	src/auth/auth.js
SQL injection	All queries use prepared statements with bound parameters	src/db/database.js
Authorisation	Role-based: cart/orders/wishlist require a valid token; admin routes require role=admin	src/api/*Routes.js
Open redirect	/go validates the destination host against an allow-list	src/store/merchantUrls.js, src/server.js
Input validation	Field and format checks (email, password length, phone, quantities, product existence)	src/api/authRoutes.js, cartRoutes.js
Payload limits	Request bodies capped (1 MB; 5 MB for catalogue import) to avoid abuse	src/api/http.js
Data exposure	Password hashes are never serialised; demo/seed accounts are hidden from user lists	src/auth/auth.js

Note: the JWT is kept in localStorage for simplicity; in a production deployment prefer httpOnly, Secure, SameSite cookies and add CSRF protection and refresh-token rotation.

13. Project Experience

Building ShopSense demonstrated how far a “zero-dependency” design can go and how classical data structures and information-retrieval algorithms combine into a real product:

- Information retrieval: implementing BM25, TF-IDF vectors, RRF and a trie from scratch made the trade-

offs between recall and precision concrete.

- Machine learning: item-item CF and SGD matrix factorization showed how implicit feedback becomes personalisation, and why cold-start needs a separate path.
- Backend engineering: hand-rolling routing, JWT auth, repository access and seeding exercised clean layering and defensive coding.
- Data engineering: deduplication, category normalisation and merges turned messy public data into a coherent catalogue of 579 products with real images.
- Product sense: explainable reasons, gated interactions, wishlist/cart/orders and admin analytics tied the algorithms to real user value.
- Testing: 65 automated tests using the built-in node:test runner keep the search, recommender, API and backend behaviour stable.

14. Testing and Evaluation

- Search: relevance strategy, typo correction, facets, price filtering, Google-style recall (tests/search.test.js).
 - Recommendations: trending, content similarity, item-item CF, matrix-factorisation scoring, explanations, cold start and seen-item exclusion (tests/recommend.test.js).
 - API: endpoint contracts for catalogue, search, products, lists and /go (tests/api.test.js).
 - Backend: auth, cart, orders, wishlist, admin, catalogue without duplicates, real images and home shelves (tests/backend.test.js).
 - Affiliate: tracking parameters and the product-link resolver (tests/affiliate.test.js).
- Current result: 65 of 65 tests passing.

15. Future Enhancements

- Vector database and neural embeddings for true semantic search.
- Real-time ingestion and a message queue for events; Redis caching of hot queries and recommendations.
- Richer personalisation: session-based/sequence models, learning-to-rank from clicks, A/B testing.
- Commerce depth: payments, inventory, returns, multi-currency and shipments.
- Content: user-generated reviews, Q&A, and image-based search.
- Operational: observability (metrics, tracing), rate limiting, and horizontal scaling behind a load balancer.

Appendix A — Source Map

Folder	Contents
src/api	HTTP route modules: routes + auth/cart/order/admin/wishlist sub-routers, shared http helpers
src/search	tokenizer, inverted index, BM25, TF-IDF vectors, trie, fuzzy matching, SearchService, RRF
src/recommend	popularity, content, item-item CF, matrix factorization, RecommendationService
src/catalog	categories, brands, home merchandising, specs and reviews builders
src/db	schema.sql, database.js (open, migrate, seed, repositories)
src/auth, src/affiliate	password/JWT security; tracking + product-link resolver
src/store, src/media	in-memory store wiring; merchant URLs; SVG product imagery

public	vanilla-JS frontend: index/search/product/list/wishlist/admin
scripts	generators, seed, db reset, affiliate link resolver
tests	node:test suites (search, recommend, API, backend, affiliate)